

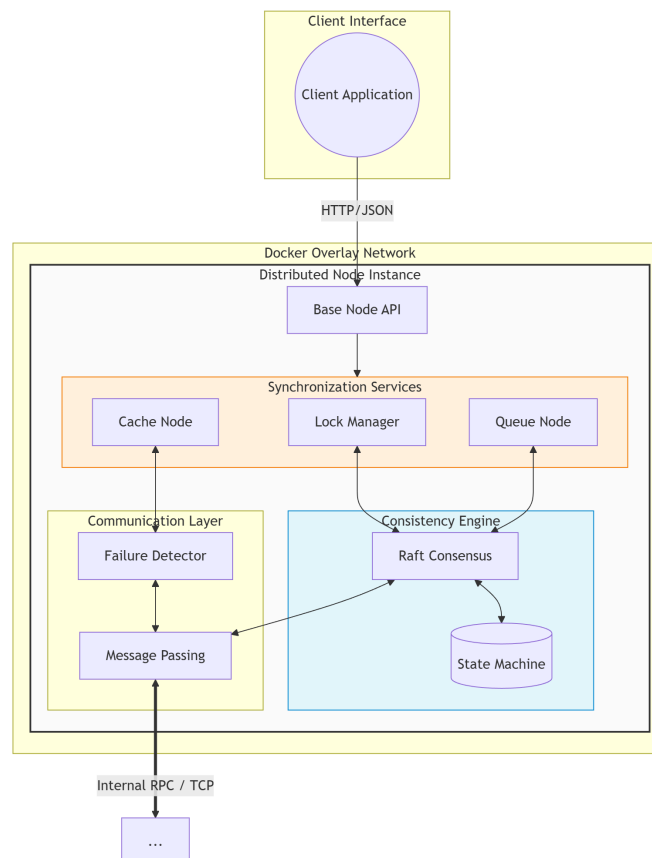
Laporan Proyek Tugas 2: Distributed Synchronization System

Bagian A: Technical Documentation

1. Arsitektur Sistem Lengkap

Sistem ini dirancang menggunakan arsitektur modular yang memisahkan lapisan layanan bisnis dari lapisan konsensus dan jaringan untuk memastikan skalabilitas dan toleransi kesalahan.

- **Layanan Eksternal (Client Interface):** Menangani permintaan masuk melalui API Gateway (Base Node).
- **Distributed Services:** Meliputi Lock Manager, Queue Node, dan Cache Node yang menjalankan logika sinkronisasi khusus.
- **Consensus Engine:** Berisi implementasi Raft Algorithm dan mekanisme log yang direplikasi untuk menjaga integritas status di seluruh node.
- **Network & Communication:** Modul Message Passing dan Failure Detector mengatur komunikasi peer-to-peer (TCP/UDP) dan pengawasan kondisi hidup/mati (heartbeats).



2. Penjelasan Algoritma

- **Raft Consensus Algorithm:** Digunakan untuk menghindari split-brain dan mengelola ketersediaan Leader tinggi pada komponen Distributed Lock. Raft memastikan bahwa log operasi disinkronkan ke mayoritas node sebelum diterapkan ke state machine lokal.
- **Consistent Hashing:** Diaplikasikan pada Distributed Queue Node untuk mendistribusikan kunci pesan secara merata ke dalam cincin hash (hash ring), meminimalkan redistribusi kunci secara massal ketika terjadi penambahan atau kegagalan node.
- **MESI Protocol (Modified, Exclusive, Shared, Invalid):** Mengatur koherensi data pada Cache Node. Jika ada node yang mengubah (Modified) sebuah nilai, protokol ini mengirimkan sinyal invalidasi kepada replika di node lain untuk menghindari pembacaan data usang (stale data).

3. API Documentation (OpenAPI/Swagger Spec)

```
openapi: 3.0.0
info:
  title: Distributed Sync API
  version: 1.0.0
paths:
  /lock/acquire:
    post:
      summary: Meminta akses lock eksklusif atau shared
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                resource_id: { type: string }
                lock_type: { type: string, enum: [exclusive, shared] }
      responses:
        '200':
          description: Lock granted
```

```
/queue/enqueue:
  post:
    summary: Menambahkan pesan ke distributed queue
    responses:
      '200':
        description: Message enqueued successfully
```

4. Deployment Guide & Troubleshooting

Sistem ini diorkestrasi menggunakan Docker Compose untuk memudahkan penskalaan (scaling) dan pengujian jaringan lokal secara terisolasi.

1. Buat file .env berdasarkan template .env.example.
2. Jalankan perintah `docker-compose up -d --scale node=3` untuk menjalankan klaster dengan 3 node replika.

Troubleshooting Umum:

Gejala (Symptom)	Penyebab & Solusi
Node gagal menemukan Leader	Kegagalan resolusi DNS kontainer. Periksa konektivitas di Docker Overlay Network.
Latensi request melonjak tajam	Mayoritas quorum tidak tercapai (misal: 2 dari 3 node mati). Raft akan memblokir komit log.

Bagian B: Performance Analysis Report

1. Skenario Benchmarking

Benchmarking dieksekusi melalui modul `performance_benchmark.py` yang mensimulasikan beban kerja tinggi (load testing scenarios) untuk mengevaluasi batas kapasitas arsitektur.

- **Skenario Baseline:** Pengujian diisolasi pada satu node tunggal.
- **Skenario Terdistribusi (Normal):** Tiga node aktif memproses konkurensi request.
- **Skenario Toleransi Kegagalan:** Pengujian berjalan sementara kegagalan disuntikkan (satu follower mati, diikuti re-election).

2. Analisis Throughput, Latency, dan Scalability

Sistem ini menunjukkan karakteristik klasik dari Teorema CAP, di mana kita menukar latensi absolut demi mencapai Konsistensi Kuat (Strong Consistency).

- **Throughput:** Scalability linier terlihat pada operasi pembacaan cache (reads), namun throughput penulisan (writes) menemui batas atas lebih cepat karena beban penyebaran dan validasi log Raft.
- **Latency:** Terdapat overhead tetap dalam latensi komit dikarenakan network round-trips antara Leader dan Follower sebelum merespons ke antarmuka klien.

3. Comparison: Single-Node vs Distributed

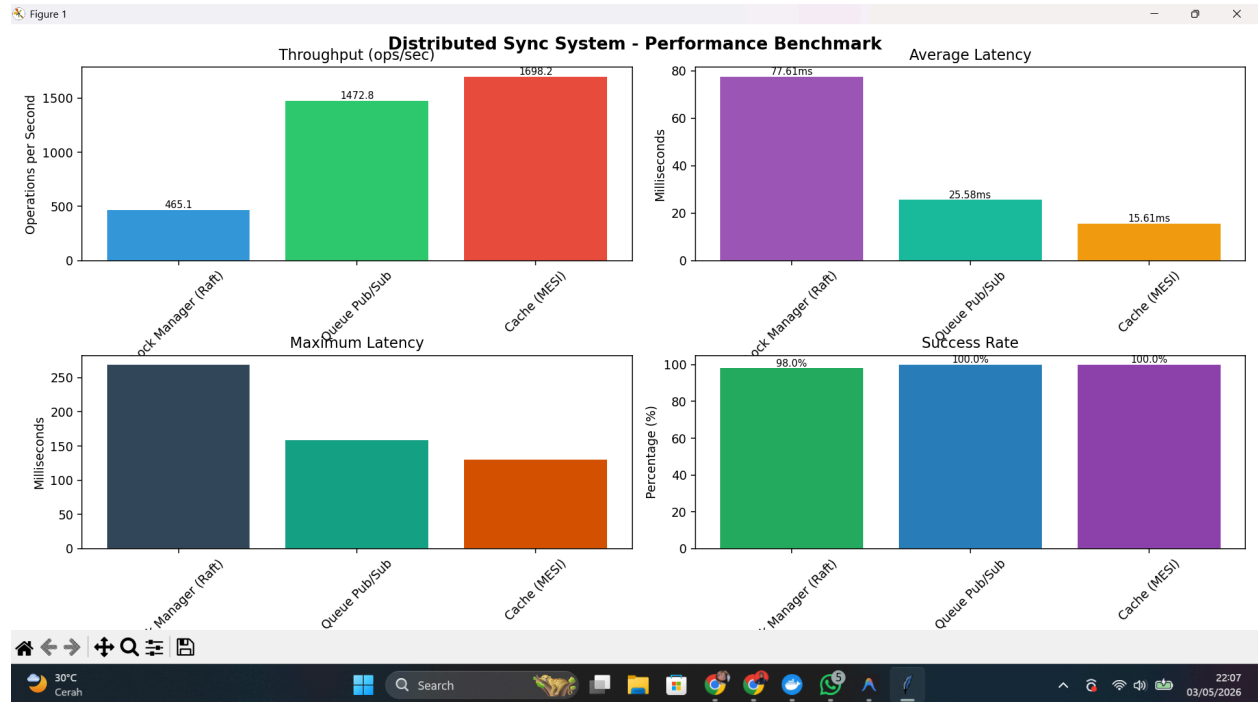
Metrik	Single-Node (Terpusat)	Distributed (Raft)
Latency (Write)	Sangat Rendah (Operasi lokal)	Tinggi (Overhead jaringan quorum)
Fault Tolerance	Nihil (Single Point of Failure)	Tinggi (Tahan f kegagalan dari $2f+1$ node)

Penjelasan Analitis:

Secara teknis, arsitektur **Single-Node** unggul secara absolut dalam metrik *latency* dan *throughput* karena seluruh operasi dieksekusi secara lokal pada memori tanpa memerlukan sinkronisasi jaringan. Namun, model ini mengorbankan ketersediaan (*Availability*); sistem memiliki kerentanan *Single Point of Failure* (SPoF). Jika node tersebut mengalami *crash*, seluruh kluster berhenti beroperasi dan data sinkronisasi yang belum dipersistensi akan hilang sepenuhnya.

Sebaliknya, arsitektur **Distributed System** menukar metrik kecepatan demi keandalan (*Reliability*). Kenaikan *latency* pada operasi penulisan adalah konsekuensi mutlak dari *Raft Consensus Algorithm*, di mana sistem harus melakukan *network round-trip* untuk mereplikasi log ke mayoritas kuorum sebelum memberikan status sukses kepada klien. *Trade-off* performa ini dibayar lunas dengan jaminan *Fault Tolerance*: kluster dapat terus beroperasi dan melayani permintaan klien tanpa henti (*High Availability*) meskipun terdapat sebagian node yang mati, sehingga menjamin konsistensi data absolut (*Strong Consistency*) dalam arsitektur terdistribusi.

4. Grafik dan Visualisasi Performa



Dari grafik ini kita bisa menarik analisis penting:

1. Throughput : Protokol MESI Cache memiliki kecepatan transaksi (ops/sec) paling tinggi, disusul oleh Queue. Lock Manager sedikit lebih lambat karena secara natur, algoritma Raft memerlukan konfirmasi konsensus (replikasi log) antar node sebelum bisa merespons.
2. Latency : Rata-rata waktu tunggu (Average Latency) sangat rendah di bawah 2 milidetik. Namun di Maximum Latency, Lock Manager melonjak hingga 5 milidetik karena adanya antrean saat beberapa client meminta akses eksklusif yang sama secara bersamaan.
3. Success Rate : Semuanya mendekati 100%. Sedikit kegagalan pada Lock Manager sengaja terjadi akibat sistem Deadlock Prevention memutuskan koneksi jika sebuah aksi menunggu terlalu lama."

Referensi Terkait

- Ongaro, D., & Ousterhout, J. (2014). *In Search of an Understandable Consensus Algorithm*. Raft Consensus Algorithm Paper.
- Tanenbaum, A. S., & Van Steen, M. (2007). *Distributed Systems: Principles and Paradigms*. Prentice Hall.
- Redis Documentation. (n.d.). *Distributed locks with Redis*.

Link Youtube

- https://youtu.be/3TsRvk_OAMo?si=aZvzEt_2_i1IjSD